
Running LLMs for Economic Experiments

Felix Drinkall · ETH Summer School in AI & Applied Economics · September 2026

LLMs in the Economics Workflow

A · Measurement

Text classification at corpus scale

Sentiment, topics and policy stance that become variables in a panel.

B · Extraction

Structured facts from documents

Fields from filings, contracts and judgments, filled into a fixed schema.

C · Simulated agents

LLMs as experimental subjects

Vignette and game experiments run on model subjects before or alongside humans.

D · Text-as-data regressions

Model internals as variables

Embeddings, log-probabilities and perplexity used directly as regressors.

All four face the same engineering decision: **how do you run the model?**

The Escalation Ladder

VRAM, rough guide

1	Prompt a frontier API Zero setup.	0 GB
2	Prompt an open-weight model locally Small-to-mid models (1–70B) that fit one card.	~16–80 GB
3	Fine-tune a small encoder (0.15B) The cheapest thing that works on labelled text.	~2–8 GB
4	LoRA-tune a mid-size open model Frozen base plus a trainable adapter.	~16–80 GB
5	Full fine-tune Only for genuine domain shift. Similar techniques to prompting the largest open models.	~80 GB–multi-node

Rule:climb only when the step below fails, and stop the moment it works.

VRAM figures are order-of-magnitude guides. The real number depends on model size, precision, sequence length and batch size; the memory slides later make it exact.

What the Hardware Costs

System	GPU memory	One-off price	Typical rental
Personal laptop (RTX 4090 mobile)	16 GB	~\$2,500	;
Used RTX 3090 (2020)	24 GB	~\$800	~\$0.20/hr
NVIDIA H100	80 GB	\$30–40k	\$2–4/hr
NVIDIA H200	141 GB	\$35–45k	~\$4.50/hr
GB200 superchip (Blackwell)	192 GB	sold per rack	\$10.50–27/hr
DGX B200 server (8× B200)	8 × 192 GB	~\$515k list	\$6–16/hr per GPU

- Everything up to QLoRA on a 13B model fits in the first three rows
- Renting an H100 for a week of fine-tuning costs about \$400, roughly one conference registration

Prices: CloudZero, getdeploying.com and Silicon Analysts market surveys, August 2026. Cloud rates are on-demand per GPU and vary by provider.

One Problem, Carried Up the Ladder

The task

The target: a tone score $\hat{y}_{it}$ for every firm-quarter, measured from 200k earnings-call transcripts (2005–2026). A text-derived panel variable for firm-level regressions, rebuilt each quarter as new calls arrive.

Step 0, before any model

Hand-label 800 passages with an RA. Fix the metric, the acceptance bar and a time split. A week of reading that judges every step.

In and out, the same at every step

Input

“Demand softened into year-end; we see continued headwinds in Europe...” a passage from firm i ’s call in quarter t

Model

Output

$\hat{y}_{it} \in [-1, 1]$ management tone: -1 pessimistic · 0 neutral · +1 optimistic

Analysis

200k-row panel: $\hat{y}_{it}$ as regressor or outcome alongside firm controls

THE ANALYSIS GROWS WITH EACH STEP

step 1

coarse tone label over the corpus

step 2

continuous, reproducible logprob index

step 3

a small local model: quarterly updates cost nothing

step 4

a new output kind: records of each call’s guidance revisions (metric, direction, size)

step 5

simulated manager responses

the output distribution is known

Each step divider returns to this example: what the last step could not do, and what the next one buys.

The Frontier API

No GPU and no ops. Working in an afternoon.

1	Prompt a frontier API	0 GB
2	Prompt an open-weight model locally	~16–80 GB
3	Fine-tune a small encoder (0.15B)	~2–8 GB
4	LoRA-tune a mid-size open model	~16–80 GB
5	Full fine-tune	~80 GB–multi-node

THE RUNNING EXAMPLE · EARNINGS-CALL TONE

The task: a management-tone panel from 200k earnings-call transcripts (2005–2026), updated quarterly

Step 0, already done: 800 passages hand-labelled with an RA; the bar and a time split fixed before any model runs

First move: the cheapest step: one pinned frontier model over the corpus

When the API Is Enough

Generic task

Sentiment, topic labels, summarisation. A frontier model is already near the ceiling.

Small sample

A few thousand documents. Token cost is trivial next to a GPU queue and a week of setup.

Hard reasoning needed

Multi-step extraction or judgement-heavy coding where small open models still fail.

Still exploring

Iterate on prompts over a subsample before committing to any pipeline.

Public text only

Filings, news, speeches and published papers carry no confidentiality constraint.

No ops capacity

No CUDA debugging, no cluster queue, no ops burden on an RA.

Experiments That Fit the API

C · Simulated agents

Vignette and game experiments on model subjects

Ultimatum games, hiring vignettes, willingness-to-pay elicitation. The treatment is the prompt, and a few thousand calls covers the design.

Example: Horton (2023), simulated labour-market vignettes with LLM subjects.

A · Measurement, small N

Labelling a few thousand public documents

Hawkish or dovish coding of central-bank statements; stance in a few thousand speeches. Validate against a hand-coded subsample and you are done.

Example: Hansen and Kazinnik (2023), classifying Fedspeak with GPT.

Any type · Prototyping

Piloting the design before any pipeline

Test the taxonomy, the prompt and the failure modes on 200 documents before you commit to a GPU, a licence or a labelling budget.

Example: Lopez-Lira and Tang (2023), prompting over news headlines before any pipeline.

Common shape: small N, public text, and the constraint is model capability rather than cost.

Running the API Like an Instrument

Practice	Why
temperature=0, fixed seed	As deterministic as the host allows; sampling noise is measurement error in your labels
Pin the dated model snapshot	The dated version, never the moving alias: models change and retire under you (the reproducibility evidence, step 2)
Structured output (JSON schema)	Labels arrive machine-readable; no cleanup pass over prose answers
Batch endpoint for corpus jobs	Half price (next slide), and real-time rate limits stop binding
Log prompt, response, version, date	The replication package for a prompt-based measurement
Hold out a labelled validation set	A prompt edit is a model change: re-validate before rerunning the corpus

Both risks are measured, not hypothetical

Prompt wording:across 11 classification tasks, trivial edits (extra whitespace, a “thank you”, a format request) shifted predictions and accuracy. [Salinas & Morstatter \(2024\), arXiv:2401.03729](#)

Temperature 0 is not determinism:10 identical temperature-0 runs still varied in accuracy by up to 15%, and no model gave repeatable outputs on every task. [Atil et al. \(2024\), arXiv:2408.04667](#)

Rule:a prompt is a measurement instrument: version it, validate it, freeze it.

What Step 1 Costs

List prices, per 1M tokens (Aug 2026)

Model	In	Out	AA Index
Claude Opus 5	\$5.00	\$25.00	63
Claude Fable 5	\$10.00	\$50.00	62
GPT-5.6 Sol (flagship)	\$5.00	\$30.00	61
Kimi K3 open	\$3.00	\$15.00	60
GPT-5.6 Terra (mid)	\$2.00	\$12.00	55
Claude Sonnet 5	\$3.00	\$15.00	55
GLM-5.2 open	\$1.40	\$4.40	51
GPT-5.6 Luna (budget)	\$0.20	\$1.20	51
DeepSeek V4 Pro open	\$0.66	\$1.98	44
Claude Haiku 4.5	\$1.00	\$5.00	30

Batch: 50% off every rate · cached prefix: 90% off · open = weights downloadable, so the same model can move to step 2 · ranked by AA Index: [Artificial Analysis Intelligence Index](#) v4.1.1, max reasoning effort, Aug 2026

Worked example: classify 1M documents (700 in / 5 out)

Model	Standard	Batch	Batch+cache
Claude Opus 5	\$3,625	\$1,813	\$1,419
Claude Fable 5	\$7,250	\$3,625	\$2,838
GPT-5.6 Sol	\$3,650	\$1,825	\$1,431
Kimi K3	\$2,175	\$1,088	\$851
GPT-5.6 Terra	\$1,460	\$730	\$573
Claude Sonnet 5	\$2,175	\$1,088	\$851
GLM-5.2	\$1,002	\$501	\$391
GPT-5.6 Luna	\$146	\$73	\$57
DeepSeek V4 Pro	\$472	\$236	\$184
Claude Haiku 4.5	\$725	\$363	\$284

Batch+cache: batch rates with a system prompt at 25% of input tokens served from cache (90% off cached reads); assumes the vendor offers both discounts. For scale: the same corpus through a local 7–8B on one rented H100 (slide 4 prices) is a tens-of-hours, order-\$100 job. The budget rows compete with self-hosting; the flagships never do.

Rule: price the corpus before you pick the step: tier choice moves the bill 50×, batch halves it, and the prompt prefix is nearly free when cached.

Sources:[OpenAI pricing](#)·[Anthropic pricing](#)· Moonshot, Z.ai, DeepSeek pages, Aug 2026. DeepSeek rows are off-peak; the two UTC peak windows double them.

Where the API Fails

Reproducibility

Weights change under you, and hosted models are withdrawn outright; a referee cannot reproduce your 2026 run in 2028.

OpenAI retired GPT-3 (text-davinci-003) after about 14 months and GPT-4.5-preview after under five; GPT-4 leaves the API in October 2026. Pin the version and assume it disappears.

Data governance

Your data leaves the building, which rules out licensed text and admin microdata.

Research-data-centre and central-bank enclave agreements bar record-level data from leaving; a single API call can breach the data-use agreement.

Cost at scale

Cost scales linearly with tokens. At millions of documents the meter becomes the constraint.

Classifying ten million earnings-call paragraphs at frontier rates is a six-figure bill; a rented H100 does the same job for a few hundred dollars.

No internals, no explanations

Text out only: no logprobs, embeddings, attention or hidden states, as variables or as evidence.

An embedding-based predictor (Chen, Kelly and Xiu, 2023) cannot be built from chat output, and a referee gets no logprob margins or token attributions behind the labels.

Multimodal data

Mixed text and numerical data: filings with balance-sheet tables, calls with price series.

Chat APIs flatten tables into lossy text; running the model yourself lets you align token embeddings with the numeric series your regression actually uses.

Any one of these means running the model yourself.

Open Weights, Your Hardware

The same prompting, on weights you control.

1	Prompt a frontier API	0 GB
2	Prompt an open-weight model locally	~16–80 GB
3	Fine-tune a small encoder (0.15B)	~2–8 GB
4	LoRA-tune a mid-size open model	~16–80 GB
5	Full fine-tune	~80 GB–multi-node

THE RUNNING EXAMPLE · EARNINGS-CALL TONE

Last step: a pinned API model scored the corpus overnight for ~\$110 and passes coarse tone

The issue: the label is a black box: no logprobs for a continuous index, the model can be retired mid-project, and referees might not be able to rerun it

This step buys: pinned local weights: a continuous, reproducible logprob tone index

What Open Weights Buy You

YOU GAIN

- Fixed, citable weights and replicable results
- Data never leaves your machine or secure enclave
- Logprobs, embeddings and hidden states
- Marginal cost near zero once running, which is the only viable route at millions of documents

YOU PAY

- VRAM: a card big enough for weights plus cache
- Setup time: environment, serving stack, quantised loading
- Debugging: CUDA versions, drivers, out-of-memory errors
- Some capability: open 1–70B models trail the frontier on hard reasoning

Rule of thumb: run the model yourself when the data cannot leave, tokens get expensive, or the paper needs weights a referee can re-run in five years.

Experiments That Need Open Weights

D · Text-as-data regressions

Model internals as regressors

Embedding distances as similarity measures, perplexity as a surprise index on announcement text, logprob margins as classifier confidence. The API never exposes these.

Example: Chen, Kelly and Xiu (2023), LLM embeddings of news as return predictors.

B · Confidential extraction

Admin microdata in a secure enclave

Extracting fields from tax records, medical notes or licensed contracts. The data-use agreement forbids any external call, so the model must come to the data.

Example: coding occupations in administrative tax data inside a research data centre.

A · Measurement, large N

Millions of documents, one classifier

Every 10-K since 1996, every earnings call, a century of newspapers. At this N the API meter dominates the budget; a local model runs at near-zero marginal cost.

Example: Bybee, Kelly, Manela and Xiu (2024), measuring the structure of economic news at WSJ scale.

Common shape: the constraint is structural. Internals, governance, or N, rather than capability.



Hugging Face: Where Open Weights Live

The Hub

Over a million open models and datasets. Every model has a card, a licence, and a dated revision hash: the thing you cite in the paper and pin in the replication package.

The libraries

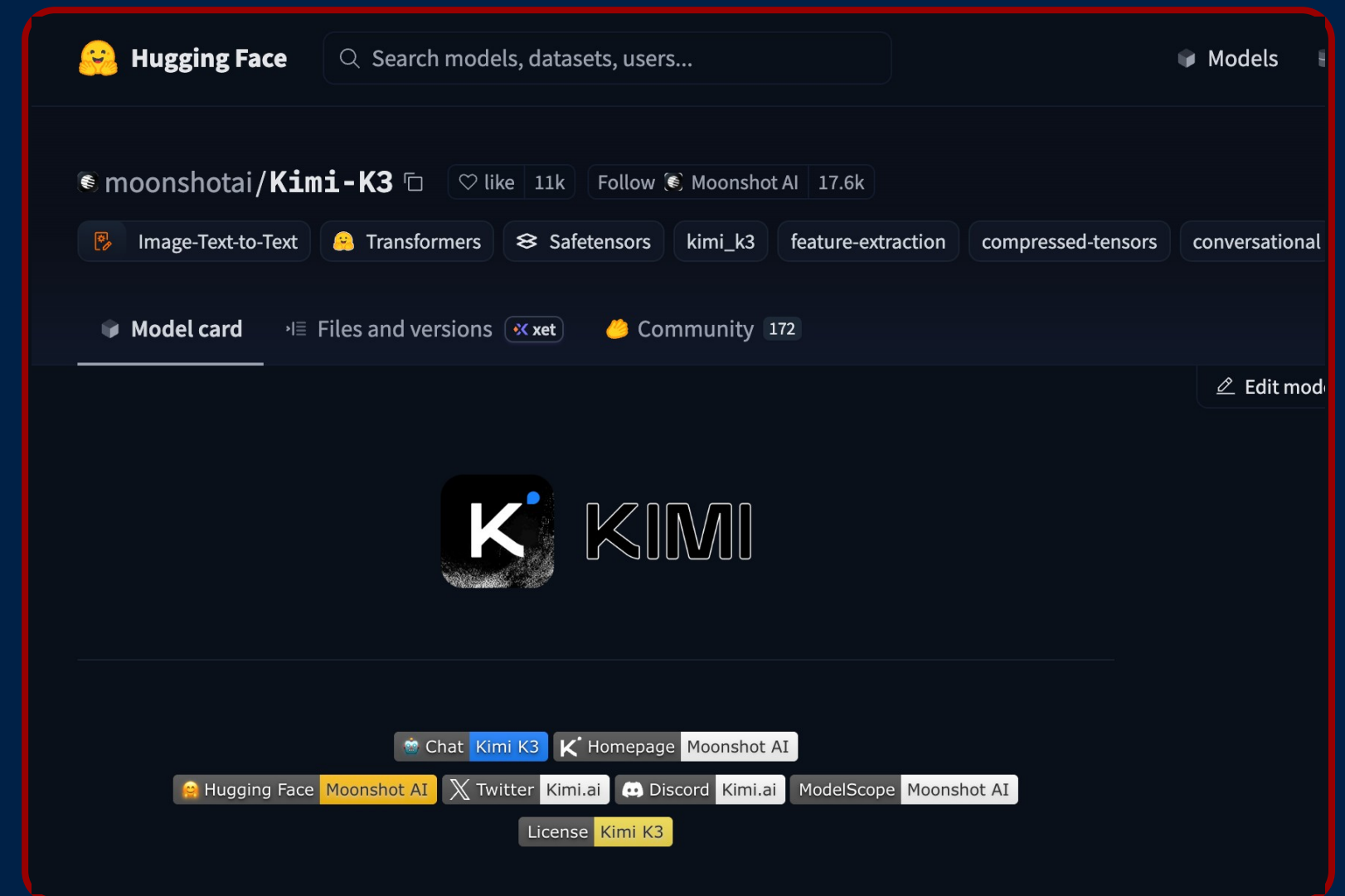
transformers for loading and inference, peft for LoRA, trl for training loops, datasets for data, bitsandbytes for 4-bit loading. Every code slide in this talk uses them.

Three checks before you commit to a model

Licence: Apache-2.0 and MIT are unrestricted; Llama-style community licences carry terms your university may need to review

Pretraining cutoff: a model trained past your sample period has seen the future, a lookahead bias no fixed seed can undo

Gated access: some models (Llama among them) require you to accept their terms on the website before downloading. Approval comes with an access key that your scripts use to download the model automatically, so the pipeline can rerun on a cluster without anyone clicking through the website again



Inference Memory: Weights + the KV Cache

Weights= parameters × bytes/param (BF16 = 2 · INT8 = 1 · NF4 = 0.5)

The model’s learned parameters: fixed numbers loaded onto the card once, the same for every document.

KV cache= 2 × layers × kv_heads × head_dim × seq_len × batch × bytes

The attention keys and values of every token seen so far, kept in memory so they are not recomputed; it grows with each token of context and each document in the batch.

Context (8B, BF16)	KV cache	+ weights
4k tokens	0.5 GB	16.5 GB
32k tokens	4.3 GB	20.3 GB
128k tokens	17.2 GB	33.2 GB
32k × batch 8	34.4 GB	50.4 GB

Worked example

8B model, BF16 weights. 32 layers, 8 KV heads (GQA), head_dim 128.

$$2 \times 32 \times 8 \times 128 \times 2 \text{ B} = 0.13 \text{ MB per token}$$

Long-document work on 10-Ks, transcripts and judgments is **cache-bound, not weight-bound** . GQA cuts the cache 4–8×, an 8-bit cache halves it again, and vLLM's paged attention stops you wasting the rest.

Bits and Bytes: Floating-Point Formats

■ sign ■ exponent (range) ■ mantissa (precision)

$-6.5 = (-1)^1 \times 1.625 \times 2^2$ sign 1 · exponent 2 sets how large · mantissa .625 sets how exact

FP32	1	8	23	32 bits = 4 bytes/param
FP16	1	5	10	16 bits = 2 bytes/param
BF16	1	8	7	16 bits = 2 bytes/param
FP8	1	4	3	8 bits = 1 byte/param
FP4	1	2	1	4 bits = 0.5 bytes/param

- FP16 halves memory but narrows the exponent: values outside its range overflow, the old loss-scaling problem
- BF16 keeps the full FP32 exponent and gives up precision instead, so training is stable at 2 bytes per parameter
- FP8 (E4M3) is the training format of H100-class tensor cores; FP4 (E2M1) is Blackwell's inference format. Their integer cousins, INT8 and NF4, are on the next slide

Bits and Bytes: INT8 and NF4

INT8

1 byte/param

- Only 256 values can be stored in 8 bits
- The weight matrix is scaled onto integers: absmax to $[-127, 127]$ or zeropoint to $[0, 255]$
- Extreme outliers in the same matrix skew the scale for everything else. `LLM.int8()` handles them by splitting them out: the few outlier dimensions (roughly 0.1%) are multiplied in 16-bit, the rest in INT8, so no precision is wasted covering their range

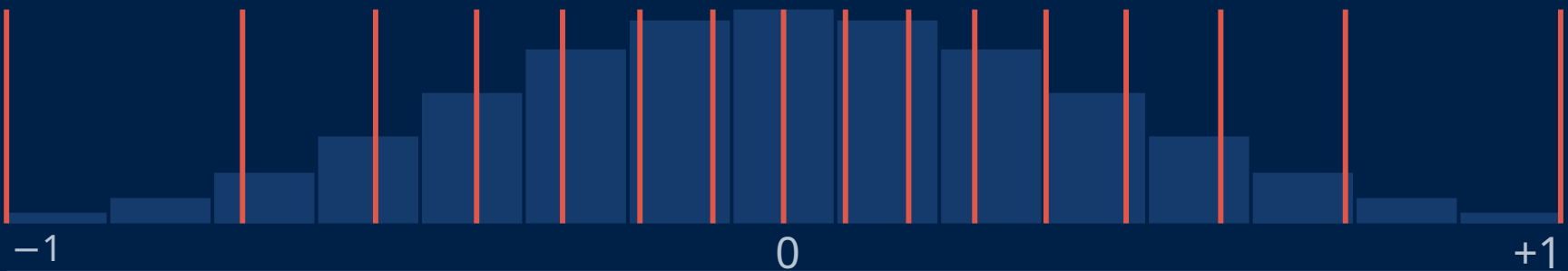
NF4

0.5 bytes/param

- 16 levels placed at the quantiles of a normal distribution, matching how trained weights are distributed
- One scaling constant per 64-weight block absorbs local outliers
- Double quantisation: the per-block scaling constants are themselves stored in FP32, one per 64 weights, which adds 0.5 bits per parameter; quantising those constants to 8-bit recovers most of that overhead

When each is used: FP formats do the maths: activations, gradients and adapters stay BF16. INT8 and NF4 only store frozen weights, INT8 to serve a model that barely fits, NF4 as the frozen base under QLoRA, dequantised back to BF16 block by block for every multiply.

Format	Bytes / param	8B model weights
FP32	4	32 GB
BF16	2	16 GB
INT8	1	8 GB
NF4	0.5	4 GB



red lines= the 16 values NF4 can store; **blue bars** = how trained weights are distributed. Every weight is rounded to its nearest red line: dense where most weights live, sparse in the tails.

Getting the Internals Out: Logprobs and Perplexity

```
from transformers import AutoModelForCausalLM, AutoTokenizer
import torch

tok = AutoTokenizer.from_pretrained(
    "meta-llama/Llama-3.1-8B" )
model = AutoModelForCausalLM.from_pretrained(
    "meta-llama/Llama-3.1-8B" ,
    torch_dtype=torch.bfloat16, device_map=
    "auto" )

enc = tok(document_text, return_tensors=
    "pt" ).to(model.device)
with torch.no_grad():
    out = model(**enc)

logprobs = out.logits.log_softmax(-1)
← token-level log-probabilities nll = -logprobs[0, :-1].gather(-1,
    enc.input_ids[0, 1:, None]).mean()
perplexity = nll.exp()
← surprise measure for the document
```

One forward pass, no training: every token's probability, and perplexity for the whole document. Embeddings get their own treatment in the encoder section.

What the Internals Give You

perplexity

A surprise index

How unexpected a statement is to a model trained before it. Score every FOMC statement against the pre-announcement baseline and regress returns on the surprise.

embedding

Distance and similarity measures

Firm-to-firm product similarity from 10-K descriptions, topic drift in speeches over time, matching treated and control documents on content.

logprob margin

Calibrated label confidence

The gap between the top two label probabilities. Weight observations by confidence, and route the uncertain tail to hand-coders instead of coding everything twice.

hidden states

Features for probes and regressions

Layer activations as controls or inputs to a small probe: recover dimensions like uncertainty or sentiment without labelling a training set for each.

Fine-Tuning a Small Encoder

When prompting fails on a held-out set: the cheapest thing that works on labelled text.

1	Prompt a frontier API	0 GB
2	Prompt an open-weight model locally	~16–80 GB
3	Fine-tune a small encoder (0.15B)	~2–8 GB
4	LoRA-tune a mid-size open model	~16–80 GB
5	Full fine-tune	~80 GB–multi-node

THE RUNNING EXAMPLE · EARNINGS-CALL TONE

Last step: pinned 8B weights gave a continuous, reproducible tone index

The issue: prompting plateaus on call-speak (“headwinds”, scripted optimism), and every quarterly update reruns a 1B-token job

This step buys: a 0.15B encoder trained on our 800 labels: corpus in ~40 min, reruns free

When Prompting Stops Working

- Accuracy plateaus below your bar on a held-out set, after real prompt iteration
- The taxonomy is bespoke: categories no pretraining corpus has seen
- The language is out of domain: historical, legal, or heavily jargonised text
- You need cheap, consistent inference over a huge corpus, where a small tuned model beats a big prompted one

Try first

Fine-tune a small encoder (0.15B)

A few hundred to a few thousand labels; trains in minutes on ~4 GB. For classification, still the strongest cost-accuracy trade.

Then escalate

LoRA-tune a mid-size open model

When the task needs generation or reasoning an encoder cannot do: 4-bit frozen base, trainable adapter, one consumer card.

The Small Encoder: What the Literature Shows

Bucher and Martini (2024)

Fine-tuned BERT-style models consistently and significantly beat zero-shot GPT-3.5, GPT-4 and Claude Opus across sentiment, approval, emotion and party-position tasks, on news, tweets and speeches. Every case study, no exception.

Cost-aware comparison (2026)

Across four benchmarks, fine-tuned BERT, RoBERTa and DistilBERT match or beat zero- and few-shot LLM prompting on fixed-label tasks, at one to two orders of magnitude lower cost and latency.

ModernBERT (2024–2025)

The encoder line is still improving: a 0.4B ModernBERT reaches 93% of Llama3-1B's MMLU with 60% fewer parameters, and fine-tuned it matches or beats classification-head baselines.

The consistent pattern

Replicated across studies (Galke and Scherp 2024; Edwards and Camacho-Collados 2024; Ziems et al. 2024): with a few thousand labels, the encoder wins on fixed-label classification. Generative models win when labels are open-ended or reasoning-heavy.

Embedding benchmarks say the same: MTEB

Classification category: compact models sit at the top alongside 8B LLM-based embedders: a 0.5B embedder averages 88.8 vs 89.8 for Qwen3-Embedding-4B, and a 1.5B sits within a point of the 8B leader (89.4 vs 90.4).

Overall: LLM-backbone embedders hold the very top, but sub-1B encoders stay within a few points at a tenth the size, and still win on some suites: multilingual-E5-large (0.56B) edges Qwen3-Embedding-8B on the 49-dataset SEA-BED suite, 73.4 vs 73.0. [MTEB leaderboard](#) · Muennighoff et al. (2023); Qwen3-Embedding report (2025); SEA-BED (2025)

For fixed-label measurement, the 0.15B encoder is not a compromise. It is the frontier of the cost-accuracy trade.

Encoder vs Decoder: Who Sees What

Encoder (BERT-style): masked-token prediction

Non-sequential: the whole sentence enters at once with ~15% of tokens masked, and the loss is computed only at the masked positions.

The [MASK] raised deposit rates



Encoder LLM E_θ : bidirectional

$h_{[MASK]} \in \mathbb{R}^d$

Unembedding W_u

$$p = \text{softmax}(W_u h_{[MASK]})$$

→ bank 0.61
lender 0.17
Fed 0.08
river 0.02

“deposit” and “rates” sit on both sides of the mask, so the distribution concentrates on the financial sense.

Decoder (GPT-style): causal attention

Each token attends only to itself and the past; the future is masked (dashed).

$$h_i = f(x_1, \dots, x_i) \quad \alpha_{ij} = 0 \text{ for } j > i$$

	The	bank	raised	deposit	rates
t=1 The		?	×	×	×
t=2 bank			?	×	×
t=3 raised				?	×
t=4 deposit					?

Row t is the mask at step t: every × to the right of the diagonal is masked, and the red-outlined ? is the token being predicted at that step. At t=2 the model sees “The bank” while “raised deposit rates” are all masked, so river bank or retail bank is still unresolved until the final tokens arrive.

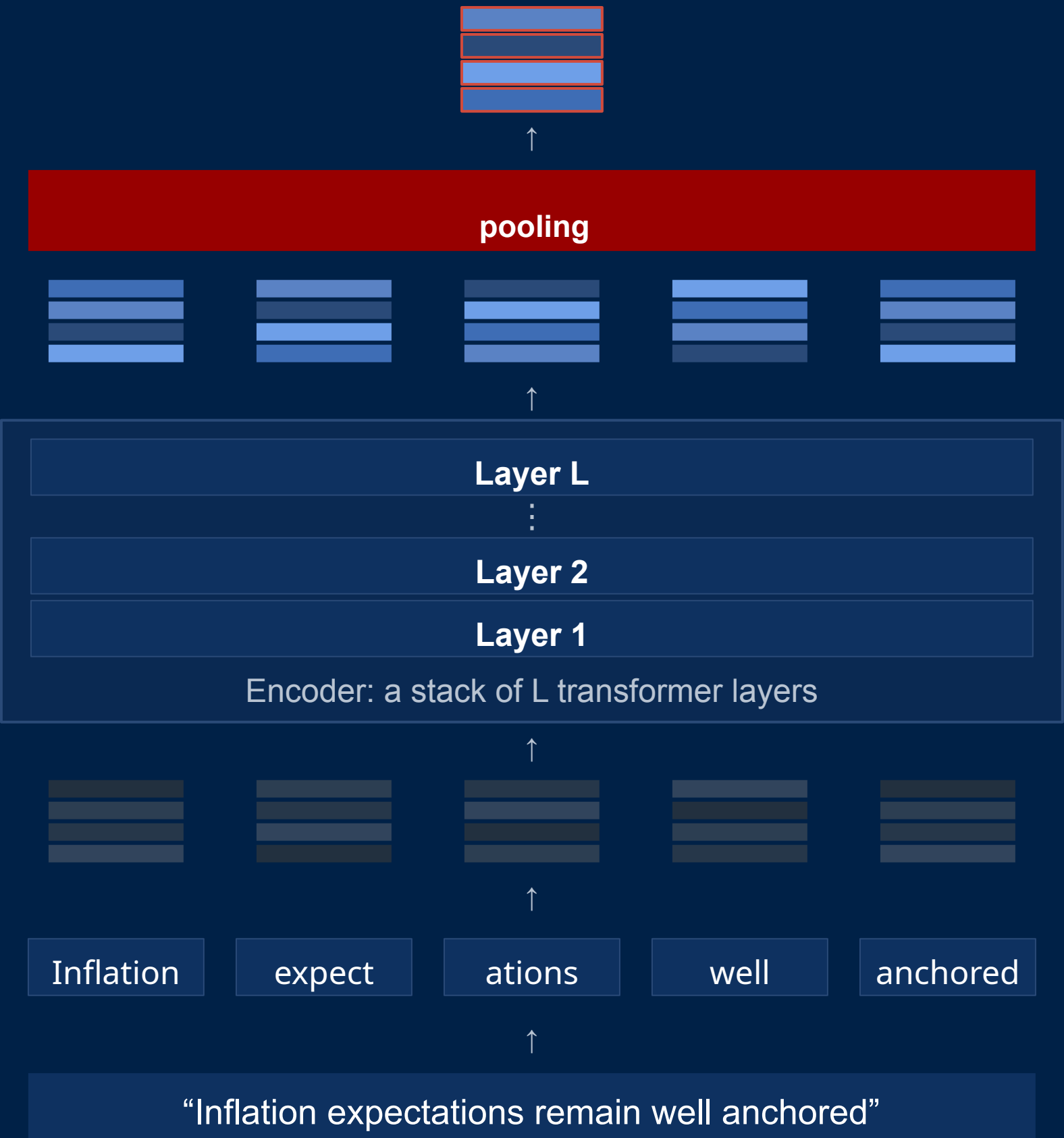
THE UNEMBEDDING MATRIX: SHARED BY BOTH

W_u projects the final vector back onto the vocabulary: decoders apply it every step for next-token probabilities (the logprobs from earlier); encoders apply it in the MLM head to predict masked tokens.

$$z_t = W_u h_t \in \mathbb{R}^{|V|}, \quad p_t = \text{softmax}(z_t)$$

Data efficiency: pretraining favours the decoder, a loss on every token against ~15% of them here (ELECTRA and ModernBERT close the gap); label efficiency at fine-tuning favours the encoder, where a few thousand labels beat prompting on fixed-label tasks.

Pooling: From Sentence to One Vector



DOCUMENT VECTOR

$$h \in \mathbb{R}^{768}$$

MEAN POOLING, OR THE LEARNED [CLS] VECTOR

$$h = (1/T) \sum_t h_t^{(L)} \text{ or } h = h_{[\text{CLS}]}$$

CONTEXTUAL VECTORS

$h_t^{(L)}$: re-written by every layer

EACH LAYER TRANSFORMS EVERY VECTOR

$$h_t^{(\ell)} = \text{Layer}_{\ell}(h_t^{(\ell-1)})$$

PRETRAINED EMBEDDING, DICTIONARY LOOK-UP

$$h_t^{(0)} = W_E[x_t]$$

TOKENISE

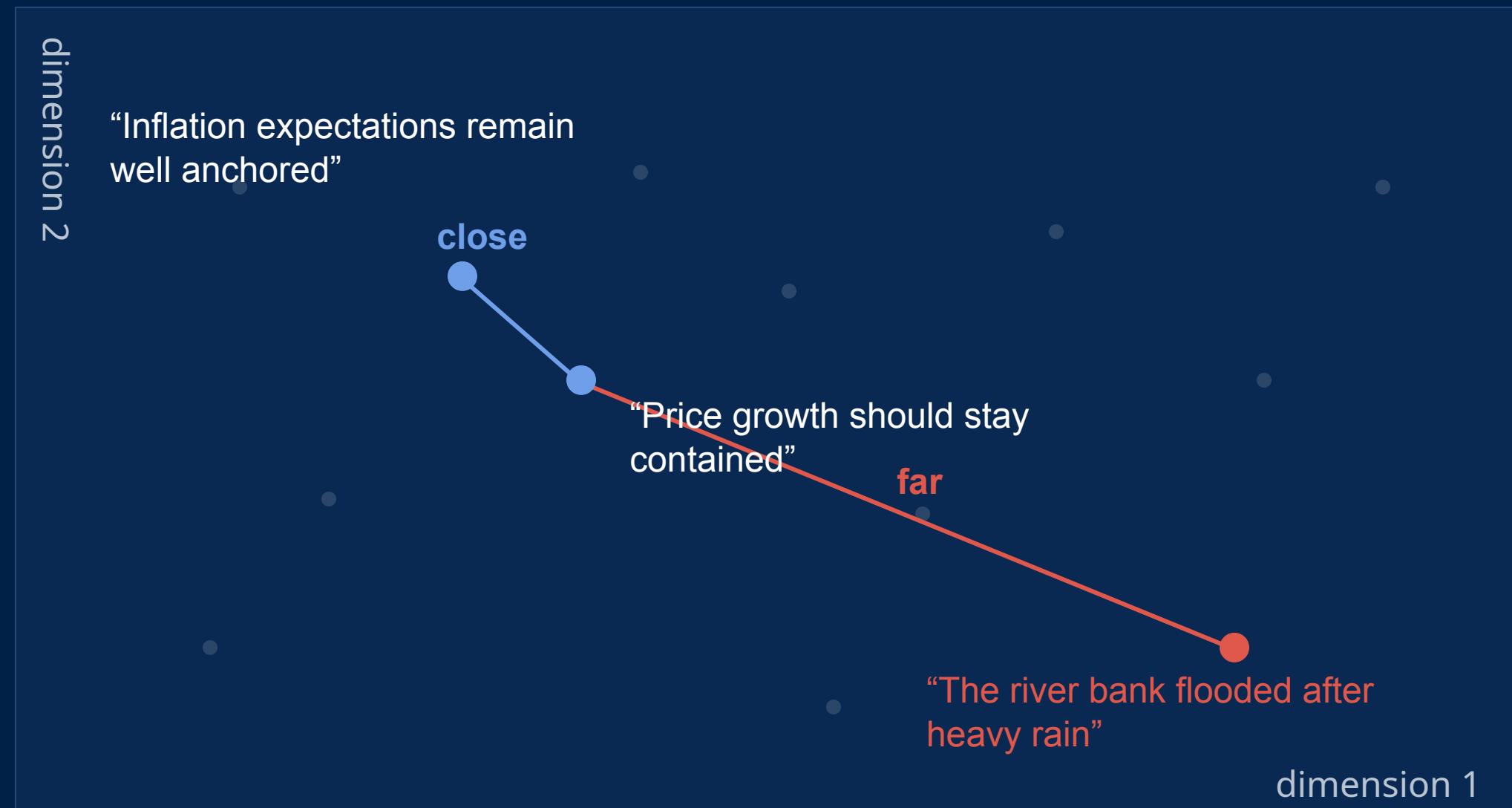
$$x = (x_1, \dots, x_T)$$

DOCUMENT

d

 = $(0.42, -1.07, \dots, 2.30)'$. Each block is a number.

Embeddings Encode Meaning, Not Words



SAME MEANING, DIFFERENT WORDS

$\|h_1 - h_2\|$ small, $\cos(h_1, h_2) \approx 1$

SHARED WORD, DIFFERENT MEANING

$\|h_1 - h_3\|$ large, despite “bank” $\approx$ “bank”

Illustrative sketch: the 768 dimensions are drawn in 2-D for the eye; distances for regressions and matching are computed in the full space. This geometry is what the similarity measures on the internals slide use.

The Encoder Task, Formally

1 • Encode

$$h_i = E_\theta(d_i) \in \mathbb{R}^d$$

The encoder E_θ maps document d_i to a d -dimensional vector ($d \approx 768$). Fine-tuning updates θ so the vector separates your labels.

2 • Classify or score

$$p(y = k \mid d_i) = \text{softmax}(Wh_i + b)_k \text{ with } \theta = \arg \min_\theta - \sum_i \log p(y_i \mid d_i)$$

$$\hat{y}_i = w'h_i + b \text{ with } \theta = \arg \min_\theta \sum_i (y_i - \hat{y}_i)^2$$

A linear head on the vector: cross-entropy for discrete labels, squared loss for a continuous score (an exposure index, a sentiment intensity).

3 • Use in the econometric model

$$y_{it} = \beta \hat{y}_{it} + \gamma' x_{it} + \alpha_i + \delta_t + \varepsilon_{it}$$

The model score enters the panel regression as a generated regressor: it is estimated, not observed, so bootstrap the full pipeline or correct the standard errors, and validate $\hat{y}$ against hand-coded labels on a held-out, time-split sample.

The Encoder in Code: Text to Vectors

```
from transformers import AutoTokenizer, AutoModel
import torch
```

```
tok = AutoTokenizer.from_pretrained(
    "answerdotai/ModernBERT-base" )
enc = AutoModel.from_pretrained(
    "answerdotai/ModernBERT-base" )
```

```
x = tok(
    "Inflation expectations remain well anchored" ,
    return_tensors=
    "pt" )
x.input_ids
```

```
with torch.no_grad():
    out = enc(**x)
```

```
out.last_hidden_state.shape
h = out.last_hidden_state.mean(dim=1)
```

← the pretrained encoder E_θ (0.15B) and its tokeniser

← tokenise: $d \rightarrow x$; each id is a row of the embedding dictionary W_E

← `tensor([[15723, 33922, 3464, 1095, 30256]])`

← inference only: no gradients are tracked, so memory is weights plus activations

← `[1, T, 768]`: one vector per token

← mean pooling: $h \in \mathbb{R}^{768}$

This is the pooling slide, executed: tokens in, one 768-dimensional vector out. No training yet.

The Encoder in Code: Labels to Task

```
labels = {"hawkish": 0, "neutral": 1, "dovish" : 2}
df[
    "y"] = df["label" ].map(labels)
data = Dataset.from_pandas(df[[
    "text", "y" ]])
data = data.map(lambda b: tok(b[
    "text" ], truncation=True), batched=True)

model = AutoModelForSequenceClassification.from_pretrained(
    "answerdotai/ModernBERT-base", num_labels=3 )

trainer = Trainer(model=model, args=TrainingArguments(
    learning_rate=
    2e-5, num_train_epochs=3, seed=42 ,
    per_device_train_batch_size=32, output_dir=
    "enc-ckpt/" ),
    train_dataset=data[
    "train"], eval_dataset=data["validation" ])
trainer.train()

logits = trainer.predict(data[
    "test" ]).predictions
probs = torch.softmax(torch.tensor(logits), -1)
yhat = probs.argmax(-1)
```

← each row pairs one text with its hand-coded label $y_i \in \{0, 1, 2\}$

←tokenise: the model receives x , never raw text; y rides along in the row

←per batch, inside the forward pass: $x \rightarrow E_\theta \rightarrow h \rightarrow (W, b) \rightarrow z$, the label only enters the loss

←minimises $-\sum \log p(y_i | d_i)$; minutes on a ~4 GB card

← $z = Wh + b$

← $p = \text{softmax}(z)$

← $\hat{y}$: the score for the regression

The full output chain: logits, probabilities, predicted label. That last column is the $\hat{y}$ in the regression.

Experiments That Justify Fine-Tuning

A · Bespoke taxonomy

Firm-level exposure measures

Political risk, climate exposure and supply-chain stress coded from earnings calls. Categories defined by your paper, learned from 2–3k labelled snippets.

Example: Hassan et al. (2019), firm-level political risk from earnings calls.

A · Domain shift

Historical and legal language

19th-century newspapers, colonial records, court judgments. Spelling, register and meaning sit far from any pretraining corpus. Genuine domain shift is the classic case.

Example: Dell et al. (2023), American Stories, structuring a century of newspapers.

B · Strict-schema extraction

Structured fields at corpus scale

Loan covenants, procurement contract terms, patent claims. A fixed schema where a QLoRA-tuned model holds format and precision over millions of documents.

Example: covenant terms from syndicated-loan contracts feeding a lending-standards panel.

Common shape: prompting failed a held-out test, and labelled examples exist or can be made.

LoRA

A low-rank adapter on a frozen base: training a generative model on one card.

1	Prompt a frontier API	0 GB
2	Prompt an open-weight model locally	~16–80 GB
3	Fine-tune a small encoder (0.15B)	~2–8 GB
4	LoRA-tune a mid-size open model	~16–80 GB
5	Full fine-tune	~80 GB–multi-node

THE RUNNING EXAMPLE · EARNINGS-CALL TONE

Last step: the encoder passes the tone bar; quarterly reruns cost nothing

The issue: the analysis grows: extract guidance changes as structured JSON. That is generation under instructions, and a classifier head cannot do it

This step buys: the 8B taught the extraction behaviour by a low-rank adapter, on the same card

When Is LoRA the Right Step?

Choose LoRA when

Prompting has plateaued on your validation set but the task is learnable from labels: domain classification, structured extraction, house style. You want behaviour changed, not knowledge added, and the budget is one 24–80 GB card.

Stay lower when

Few-shot prompting already passes the validation set (step 1–2), or a small encoder plus a linear head solves it at a fraction of the compute (step 3).

It will not save you when

The task needs new knowledge at corpus scale: continued pretraining on domain text, or the large-data regimes where adapters fall behind. That is step 5.

Rule: fine-tune behaviour with adapters; buy knowledge with data and full training. The evidence for these boundaries follows in this section.

Training Memory: Four Terms

Total VRAM $\approx$ Weights + Gradients + Optimiser state + Activations

Weights

2 B / param

The model itself: BF16 stores every parameter in 2 bytes. LoRA freezes them; QLoRA holds them in 4-bit NF4 at 0.5 B.

Gradients

2 B / trainable

One value per trainable parameter, at weight precision. PEFT makes this term almost vanish: under 1% of parameters train.

Optimiser state

2–4 B / trainable

Adam keeps two running averages, m and v (4 B); momentum-only optimisers such as Muon keep one (2 B). Still the largest single term in a full fine-tune.

Activations

batch $\times$ seq

Intermediate tensors held for the backward pass. Scales with batch and context rather than model size; gradient checkpointing trades compute to shrink it.

Full fine-tune in BF16: $2 + 2 + 2\text{--}4 = \mathbf{6\text{--}8 \text{ bytes per parameter}}$, before activations.

Optimiser States: Adam and Its Successors

AdamW: two buffers

4 B / param

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

Two running averages per trainable parameter: the first moment m (direction) and second moment v (scale). In BF16 that is 2 + 2 bytes, which is why the tables so far charged 4 B.

[Kingma and Ba \(2015\), “Adam”, arXiv:1412.6980](#) · [Loshchilov and Hutter \(2019\), “AdamW”, arXiv:1711.05101](#)

Muon: one buffer

2 B / param

$$B_t = \mu B_{t-1} + g_t$$

$$W \leftarrow W - \eta \cdot \text{NewtonSchulz}(B_t)$$

Momentum only: one buffer, half of Adam's state. Muon orthogonalises the momentum update for weight matrices and matches or beats AdamW in compute efficiency at scale; it pretrained Moonlight and the trillion-parameter Kimi K2. Lion (2023) is the earlier one-buffer sign-momentum optimiser.

[Jordan et al. \(2024\), “Muon”](#) · [Liu et al. \(2025\), “Muon is Scalable for LLM Training”, arXiv:2502.16982](#) · [Chen et al. \(2023\), “Lion”, arXiv:2302.06675](#)

Per trainable parameter: 4 B under Adam, **2 B under Muon** . On a 7B full fine-tune that is 28 GB down to 14 GB of optimiser state.

Where the Memory Goes

Term	Bytes / param	Full FT, 7B	LoRA r=16	QLoRA 4-bit
Base weights	2 · 0.5 in NF4	14 GB	14 GB	3.5 GB
Gradients	2 · trainable only	14 GB	~0.05 GB	~0.05 GB
Optimiser state (Muon)	2 · trainable only	14 GB	~0.05 GB	~0.05 GB
Activations	batch × seq	5–20 GB	5–20 GB	5–20 GB
Total		~47–62 GB	~20–35 GB	~9–24 GB

A 7B full fine-tune does not fit on one 80 GB card. The same job under QLoRA fits on a**24 GB consumer card** . That single row is the practical case for PEFT.

PEFT in 2026: What Survived

LEARN THESE

LoRA · the default. Low-rank update, frozen base, under 1% trainable parameters.

QLoRA · LoRA over a 4-bit NF4 base. The method that puts a 70B model on one card.

DoRA / QDoRA · splits the update into magnitude and direction; closes most of the remaining gap to full fine-tuning.

4-bit quantisation (NF4) · how you load anything large. BF16 is simply the default now.

Frozen layers · the idea underneath all PEFT: most weights never need to move.

KNOW THEY EXIST, DO NOT USE THEM

Adapter modules (Houlsby 2019) · the ancestor of LoRA. Lineage rather than practice.

Prompt / prefix / soft tuning · superseded by in-context learning with a good prompt.

BitFit · elegant, academic, unused.

LLM.int8() · historically important outlier story; 4-bit NF4 replaced it in practice.

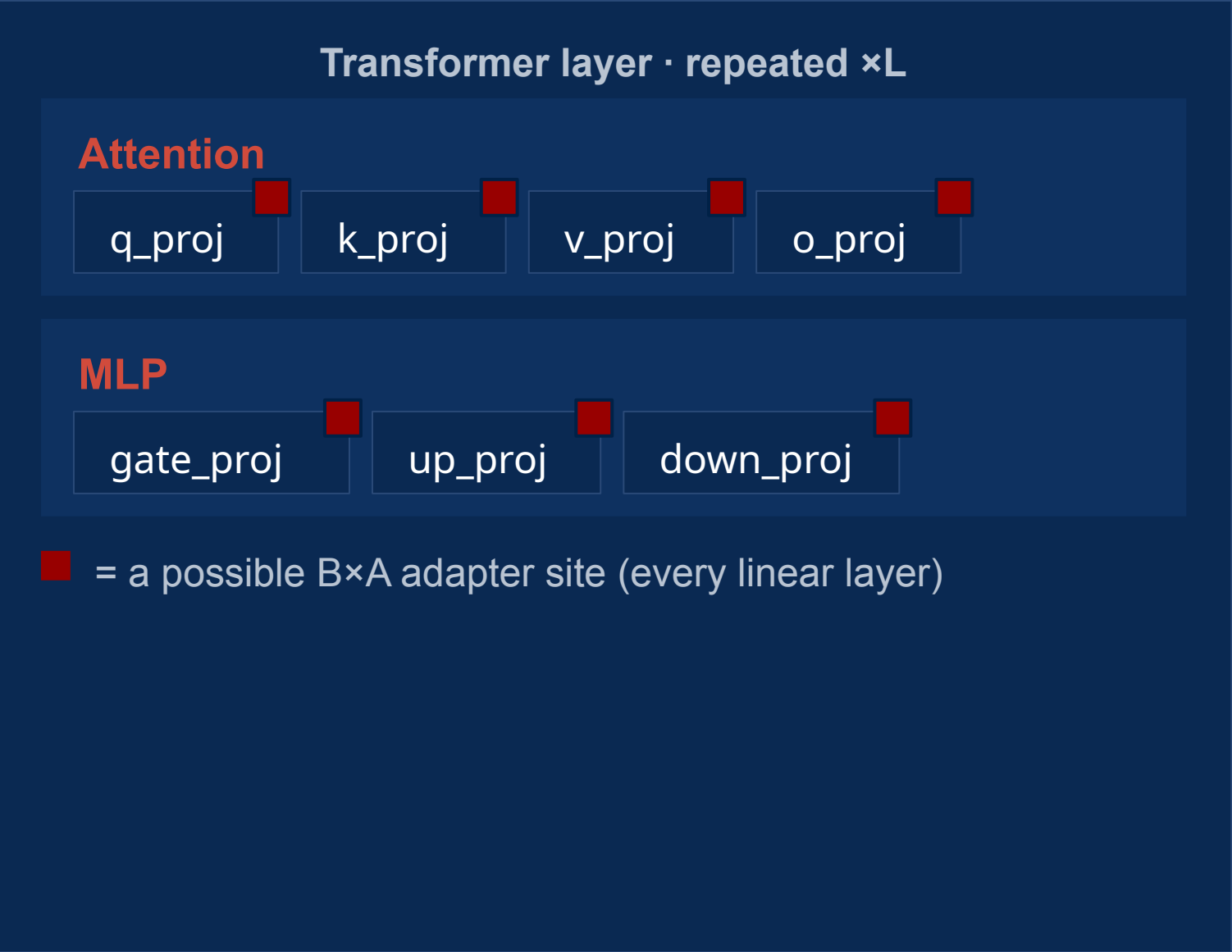
FP16 mixed-precision tricks · BF16 made the loss-scaling problem go away.

LoRA: Low-Rank Adaptation

The diagram illustrates the LoRA adaptation formula. It shows a large blue square representing the frozen weight matrix W with dimensions d (height) and k (width). This is added to the product of a vertical red rectangle B (dimensions $d \times r$) and a horizontal red rectangle A (dimensions $r \times k$). The result is a dashed blue square representing the fine-tuned weight matrix $\tilde{W}$ with dimensions $d \times k$.

- Fine-tuning targets $\tilde{W} = W + BA$: W stays frozen, only B and A train
- The rank r is the thin shared dimension of B ($d \times r$) and A ($r \times k$): 16–32 is normal, under 1% of parameters
- Gradients and optimiser state exist only for the adapter, so the two big training terms vanish
- Adapters merge into W at inference: zero added latency

Using LoRA: Placement and Settings



Tunable parameter	What it controls	Typical
r	capacity of the update: the thin B×A dimension. 16–32 covers a few thousand labels; 64–256 for instruction-tuning scale	16–32
lora_alpha	scales the update by α/r before it is added to W: higher = stronger adapter	$2 \times r$
lora_dropout	regularises the adapter when the label set is small	0.05
target_modules	which linear layers get adapters (the choice on the left)	all linear
learning_rate	adapters train hot: roughly 10× the rate of a full fine-tune	$2e-4$

Attention only (q, v)

The original recipe: enough for style, output format and light classification. Smallest adapter, least memory. Reusing full-FT learning rates is the classic way to make it look bad.

[Hu et al. \(2021\), arXiv:2106.09685 · adapters on q and v only](#)

All modules, MLPs included

The evidence-based default when the task adds knowledge, domain language or reasoning. The MLP adapters do most of the work: MLP-only nearly matches all-modules; attention-only falls short.

[Dettmers et al. \(2023\), arXiv:2305.14314 · all linear layers · Thinking Machines \(2025\), “LoRA Without Regret” · MLP vs attention ablation](#)

The payoff: with these settings right, hot rate, all modules, enough rank, LoRA matches full fine-tuning at post-training data scales. It falls behind only at pretraining-like scales. “LoRA Without Regret”, Thinking Machines Lab, 2025; consistent with Biderman et al. 2024.

Evidence I: PEFT vs Full Fine-Tuning

Average score	Prompt tuning	Prefix	LoRA	Full FT
Classification (6 task families)	68.0	76.6	78.1	78.6
All 100 tasks, incl. QA + generation	31.7	36.1	38.2	41.0
Trainable parameters (T5-base)	0.02M	17.5M	0.84M	220M

- On classification, which covers most econ measurement, LoRA sits within 0.5 points of full fine-tuning at 0.4% of the parameters
- The gap is real for open-ended QA and generation; full fine-tuning also needs ~4× the VRAM at low batch sizes

"Parameter-efficient fine-tuning of large-scale pre-trained language models", Ding et al., Nature Machine Intelligence 2023, 100 NLP tasks

Evidence II: Where LoRA Falls Short

The gap is real

- At common ranks (16–64), LoRA substantially underperforms full fine-tuning on code and maths, and the gap widens with more training data
- In continued pretraining (~20B tokens) the gap does not close even at high rank
- In instruction fine-tuning, rank 256 can match full fine-tuning; the ranks people actually use fall short
- Mechanism: full fine-tuning learns weight perturbations of rank 10–100× a typical LoRA config

The compensating virtue

- LoRA forgets less: it preserves base-model performance outside the target domain better than full fine-tuning
- It regularises better than weight decay or dropout and keeps generations more diverse
- For econ pipelines this matters: a classifier that also keeps its general language ability degrades more gracefully on odd documents

Read for our setting: LoRA is weakest when you try to teach a model a large new domain. Task adaptation on thousands of labels, the typical econ case, sits well inside its comfort zone.

"LoRA Learns Less and Forgets Less", Biderman et al., TMLR 2024. Llama-2 7B, code and mathematics.

QLoRA: LoRA on a Quantised Base

The precision story from the bits-and-bytes slides, applied to storage: the frozen base drops from BF16 to 4-bit NF4, the adapter keeps BF16's full exponent range for training. That is all QLoRA is: LoRA plus quantisation.

NF4 base

The frozen base loads in 4-bit NormalFloat: 0.5 bytes per parameter, information-optimal for normally distributed weights.

BF16 adapters

Adapters train in BF16 on top; compute dequantises blockwise on the fly.

Paged optimiser

Optimiser state pages to CPU RAM on memory spikes, so long sequences do not kill the run.

24 GB

fine-tunes a 7–13B model on one consumer card

48 GB

reaches 30–34B

1 × 80 GB

puts a 65–70B model in reach

The evidence: training in 4-bit is safe

- NF4 QLoRA matches 16-bit full fine-tuning and 16-bit LoRA on GLUE, Super-NaturalInstructions and MMLU, up to 65B
- The BF16 adapter learns to compensate the quantisation error of the frozen base

The caveat: serving in 4-bit can bite

- Long-context benchmarks: NF4 loses ~6.9% on average vs BF16, one 70B case drops 59%; Llama-3.1 8B loses ~10%
- 8-bit is nearly lossless (within 0.2–0.8% of BF16): serve long-document econ work in 8-bit or BF16, and report the precision

“QLoRA: Efficient Finetuning of Quantized LLMs”, Dettmers et al., 2023; “Does quantization affect models’ performance on long-context tasks?”, Mekala et al., 2025.

The Fine-Tuning Pipeline

1

Model + tokeniser

From the Hugging Face hub. Check the licence and the pretraining cutoff.

2

Labelled dataset

The real work: a few hundred to a few thousand examples, split by time.

3

Quantised loading

bitsandbytes NF4 puts the frozen base onto your card.

4

PEFT configuration

LoRA or QLoRA: rank, alpha, target modules. Rank 16–32 is normal.

5

Training loop

TRL / SFTTrainer or Axolotl. Learning rate, epochs, checkpointing, a fixed seed.

6

Evaluation + serving

Held-out test set against a cheap baseline. vLLM for the full corpus.

QLoRA in Code: Loading the 4-Bit Base

```
from transformers import AutoModelForCausalLM, AutoTokenizer, BitsAndBytesConfig
import torch

bnb = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type=
    "nf4",          ← information-optimal for normal weights    bnb_4bit_compute_dtype=torch.bfloat16, ← maths still runs in BF16
    bnb_4bit_use_double_quant=True,      ← quantise the quantisation constants )

model = AutoModelForCausalLM.from_pretrained(

    "meta-llama/Llama-3.1-8B", quantization_config=bnb, device_map="auto" )
tok = AutoTokenizer.from_pretrained(
    "meta-llama/Llama-3.1-8B")
```

The 8B base now occupies about **5.5 GB** of VRAM. Record the exact model revision here for the replication package.

QLoRA in Code: Adapter and Training

```
from peft import LoraConfig, get_peft_model, prepare_model_for_kbit_training
from trl import SFTTrainer, SFTConfig

model = prepare_model_for_kbit_training(model)
lora = LoraConfig(r=
16, lora_alpha=32, lora_dropout=0.05,
    target_modules=[
        "q_proj", "k_proj", "v_proj", "o_proj",
        "gate_proj", "up_proj", "down_proj"
    ]) ← all modules, MLPs too
model = get_peft_model(model, lora)
model.print_trainable_parameters()
← ~0.5% of the 8B total
trainer = SFTTrainer(model=model, train_dataset=train_ds,
    args=SFTConfig(learning_rate=
2e-4, ← ~10× a full fine-tune    num_train_epochs=3, per_device_train_batch_size=4,
    gradient_checkpointing=True, bf16=True, seed=
42,
    output_dir=
"checkpoints/" ))
trainer.train()
```

The three highlighted choices come straight from the evidence slides: high learning rate, all modules targeted, rank matched to a few thousand labels.

QLoRA in Code: Merge, Evaluate, Serve

```
model.save_pretrained("adapter/")      ← a few hundred MB: publish it from peft
import PeftModel
base = AutoModelForCausalLM.from_pretrained(
    ← reload base in BF16 to serve "meta-llama/Llama-3.1-8B", torch_dtype=torch.bfloat16)
merged = PeftModel.from_pretrained(base,
    "adapter/").merge_and_unload()
merged.save_pretrained(
    "model-merged/") ← evaluate on the time-split held-out set before touching the corpus,
# then serve millions of documents:
$ vllm serve model-merged/ --max-model-len 32768
```

Serving runs in BF16 here. Per the quantisation evidence, drop to 8-bit if VRAM demands it, and avoid 4-bit serving for long documents.

Full Fine-Tuning

Only for genuine domain shift, and never on one card.

1	Prompt a frontier API	0 GB
2	Prompt an open-weight model locally	~16–80 GB
3	Fine-tune a small encoder (0.15B)	~2–8 GB
4	LoRA-tune a mid-size open model	~16–80 GB
5	Full fine-tune	~80 GB–multi-node

THE RUNNING EXAMPLE · EARNINGS-CALL TONE

Last step: the QLoRA-tuned 8B extracts guidance changes cleanly	The issue: the question changes: simulate how managers answer analyst questions. That needs two decades of call language as knowledge, and at corpus scale adapters fall behind	This step buys: continued pretraining on 3B tokens of calls, on a node
--	--	---

When Is Full Fine-Tuning Justified?

Escalate to step 5 when

The adapter evidence breaks down: corpus-scale continued pretraining, whole-distribution shift into new domain language, the large-data regimes where LoRA falls behind. Or the last point of accuracy matters and you have the node time.

Stay at step 4 when

The task is classification, extraction or instruction-following at ordinary data scales: the parity evidence says the adapter matches the full run while training ~1% of the weights.

This section is also the pre-training stack

Full pre-training uses exactly this machinery (the parallelism, the sharding, the interconnects) at 1000× the scale. Necessary knowledge for reading the systems literature; not an advised research strategy. Frontier runs are eight-figure; continued pretraining of a 7–8B model is the realistic academic ceiling.

Rule: full fine-tuning is earned by evidence of a gap, not by ambition.

The Problem: One Model, Many GPUs

The state does not fit

Full fine-tuning carries 6–8 bytes per parameter: ~50 GB for a 7B model, ~0.5 TB for 70B, before activations, and open models now reach 2.8T parameters (Kimi K3). Cards top out at 80–192 GB.

The links are not free

Splitting work across GPUs means traffic between them, and the links differ by 30× (PCIe ~64 GB/s vs NVLink 1.8 TB/s). The link, not the GPU, often decides the method.

EVERY SOLUTION IS ONE OF THREE MOVES

1 Replicate

Copy the whole model to every GPU and split the data between them. Cheap on communication; needs the model to fit on one card. Data parallelism.

2 Shard the state

Each GPU permanently stores one slice of the weights, gradients and optimiser, gathering what it needs just in time. ZeRO / FSDP.

3 Split the model

Cut individual matrices across GPUs or the layer stack into stages. The frontier move. Tensor and pipeline parallelism.

One slide on each, then the table that says which moves your hardware can afford.

Data Parallelism in Depth



every GPU: a full copy · batch split N ways · gradients averaged once per step

The one synchronisation

After each backward pass the GPUs average their gradients: $g = (1/N) \sum_k g_k$. That is one gradient-sized message per step, about 14 GB for a 7B model, and it overlaps with the backward pass.

Why it tolerates slow links

One sync per step, not per layer: the only parallel method that stays efficient over PCIe. Throughput scales almost linearly with the number of GPUs.

Gradient accumulation

Run several micro-batches before syncing: effective batch = per-GPU batch × N GPUs × accumulation steps. Big-batch training without big-batch memory.

The requirement

Model, gradients and optimiser must fit on ONE card, since every card holds everything. This is the natural partner of encoder fine-tunes and LoRA jobs, and of large-scale inference over a corpus.

Rule: if the job fits on one card, data parallelism is how you use the other cards.

Sharded Data Parallelism: ZeRO and FSDP



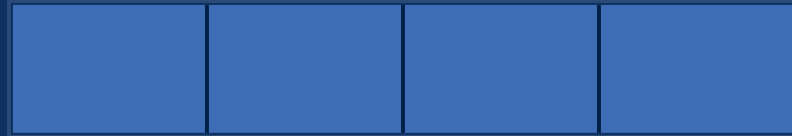
Stage	What is sharded across the N GPUs	Memory / GPU (7B)
ZeRO-1	optimiser state only	~44 GB
ZeRO-2	optimiser state + gradients	~33 GB
ZeRO-3 / FSDP	optimiser state + gradients + the weights themselves	~10-25 GB

- Per-GPU state shrinks toward $(2 + 2 + 2)/N$ bytes per parameter, plus activations
- The price: all-gather and reduce-scatter inside every layer, every step; this method wants NVLink-class bandwidth

Rule:FSDP/ZeRO-3 plus gradient checkpointing is the workhorse of full fine-tuning on one node.

Tensor and Pipeline Parallelism

Tensor parallel: split every matrix



Each weight matrix is cut across GPUs and every layer's output needs an all-reduce, forward and backward. Communication inside every matmul: NVLink only, within one node. This is how a single model too big for one card computes at all.

Pipeline parallel: split the layer stack



Consecutive layers live on different GPUs (or nodes) and micro-batches stream through the stages. Communication is small (activations at stage boundaries) but the pipeline “bubble” idles GPUs at each batch's start and end. Used across nodes at 70B+.

Frontier labs combine all three (“3-D parallelism”). For applied econ work: recognise them, rarely run them.

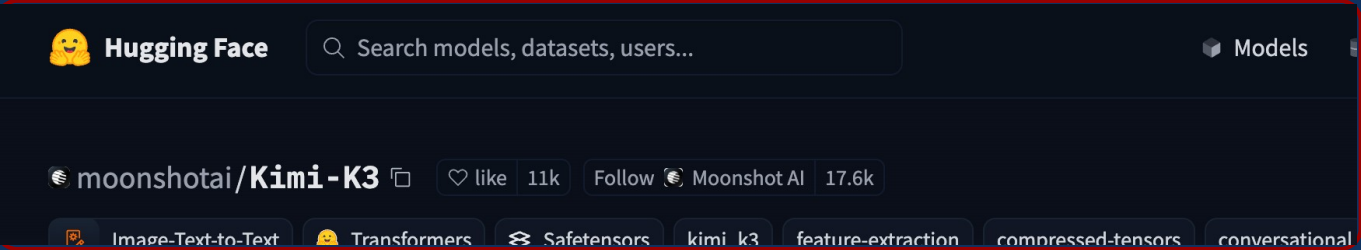
Interconnects Decide What Is Feasible

Link	Bandwidth	Methods that run well	Experiments it supports
PCIe 5.0 ×16	~64 GB/s	Data parallel only: one sync per step	Encoder fine-tunes and LoRA/QLoRA per card; corpus inference. FSDP crawls here
NVLink (H100 node)	900 GB/s	FSDP/ZeRO-3, tensor parallel within the node	Full fine-tuning up to ~70B on one 8-GPU node
NVLink 5 / NVL72	1.8 TB/s	Tensor parallel across a 72-GPU domain	Frontier-scale training and trillion-parameter inference
InfiniBand (between nodes)	~50–100 GB/s	Data parallel and pipeline stages across nodes	Multi-node runs: FSDP inside each node, DP between them

Rule: match the method to the slowest link it must cross. On PCIe, stay data parallel; save sharding and tensor splits for NVLink.

The Scale Problem: Models Have Outgrown the Card

Kimi K3: the model from our Hugging Face screenshot



2.8T parameters , mixture of experts: 896 experts, 16 active (~104B) per token

All experts must be loaded : the 4-bit (MXFP4) checkpoint is 1.56 TB

~1.7 TB VRAM floor in vLLM, before KV cache for its 1M-token context

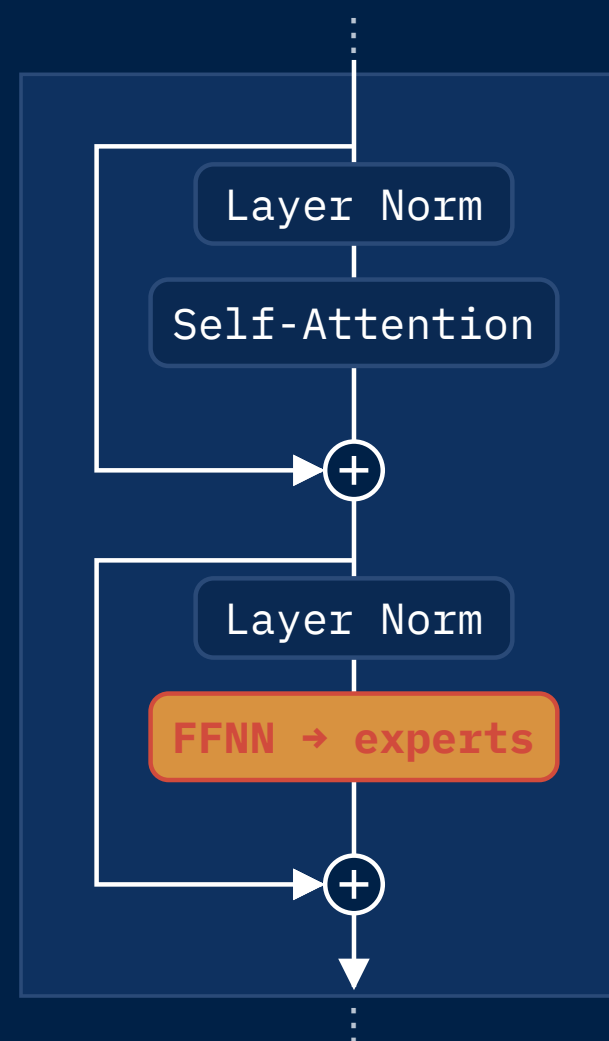
Quantisation-aware trained : the 4-bit weights are the native format, not an afterthought

Hardware	Memory	Verdict
B300, the largest single GPU (Blackwell Ultra, 2026)	288 GB	holds ~1/6 of the model
8× H100 node	640 GB	not close: cut precision + context, experimental only
8× B200 node	1.5 TB	just short of the vLLM floor
8× B300 node (HGX/DGX B300)	2.3 TB	fits, with KV-cache headroom

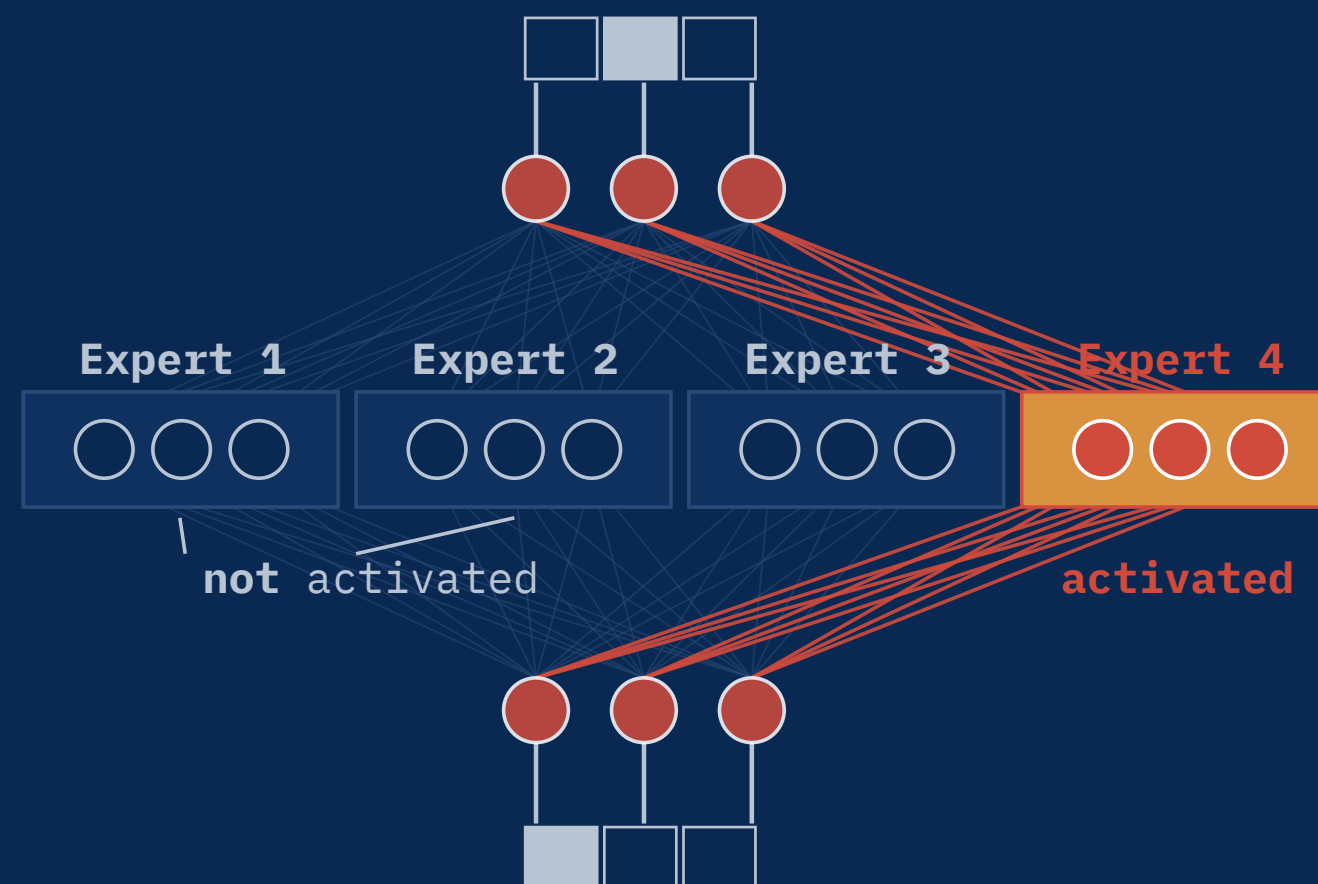
Even the biggest card on the market holds a sixth of one open frontier model. Everything in this section (the parallel methods, the interconnects, the nodes) applies to **inference as well as fine-tuning**: it is how a model larger than any card runs at all.

Sources: Moonshot AI Kimi K3 release (July 2026); vLLM model metadata; NVIDIA B300 specifications. Figures as of August 2026.

Mixture of Experts: Sparse Compute, Dense Memory



Sparse Model · the expert bank sits in the FFNN slot; router picks 16 of 896 in K3



How much of an LLM is MLP?

per layer, hidden size d :

attention $\approx 4d^2$

MLP $\approx 8d^2$

So the MLPs are $\approx 2/3$ of every layer, roughly 60–70% of a dense model. MoE multiplies exactly that block: in K3 the expert MLPs are over 95% of all 2.8T parameters.

Compute scales with ACTIVE parameters

Each token touches 16 experts: $\sim 104\text{B}$ of K3's 2.8T parameters. FLOPs per token are those of a $\sim 100\text{B}$ dense model; that is what makes frontier quality affordable.

Memory scales with TOTAL parameters

The router's pick changes every token, so all 896 experts must already sit in VRAM: 1.56 TB at 4-bit. You pay compute for 104B and memory for 2.8T.

Why not offload the idle experts?

Push them to disk or ordinary RAM and every routing miss crosses a slow link: generation drops 10–100 $\times$. Offload only works when the link is fast, which is where the new nodes come in.

The asymmetry to remember: MoE trades memory for compute, and memory is exactly what single GPUs lack.

Grace Blackwell: CPU Memory Joins the Pool

Blackwell Ultra GPU
288 GB HBM3e · 8 TB/s

NVLink-C2C

900 GB/s · cache-coherent

Grace CPU (72 Arm cores)
~480 GB LPDDR5X

One address space

GPU and CPU memory are cache-coherent: ~768 GB addressable per superchip, no explicit copies. The GB300 NVL72 rack pools 72 GPUs' worth.

What spills there

KV cache for million-token contexts, optimiser state during fine-tuning, overflow MoE experts: the bulky, less-hot parts of the memory bill.

Why it works: the link

900 GB/s is 14× PCIe. The same offload that runs 10–100× slower over PCIe becomes a usable memory tier here.

The section's lesson in one line:capacity is cheap; the connection is what makes it usable.

The Hugging Face Call for Each Method

Method	The call	What happens	Docs
Data parallel (DDP)	<code>torchrun --nproc_per_node=8 train.py</code>	Trainer detects the 8 processes and wraps the model in DDP itself: no code change	docs 
Sharded: FSDP / ZeRO	<code>accelerate launch --config_file fsdp.yaml</code>	Same script, new launcher: the config file picks the ZeRO stage. Also: <code>TrainingArguments(fsdp="full_shard")</code>	docs 
Tensor parallel (transformers)	<code>from_pretrained(..., tp_plan="auto")</code>	Shards each matrix over the GPUs (supported models); launched with <code>torchrun</code>	docs 
Tensor parallel (inference)	<code>vllm serve ... --tensor-parallel-size 8</code>	One flag splits the model across the node for serving	docs 
Layer split (naive pipeline)	<code>from_pretrained(..., device_map="auto")</code>	Accelerate places consecutive layers on successive GPUs (and CPU). No micro-batching: GPUs take turns, so it fits big models rather than speeding them up	docs 
CPU offload (the Grace story)	<code>vllm serve ... --cpu-offload-gb 200</code>	Spills weights to CPU memory, reloaded on the fly each forward pass: the docs themselves warn it needs a fast CPU–GPU interconnect	docs 

Every method on the previous slides is one launcher or one argument; the parallelism choice never rewrites your script.

What Modern GPUs Change

FP8 tensor cores

H100 onward trains in mixed FP8/BF16 at roughly twice BF16 throughput, with per-tensor scaling handled by the Transformer Engine rather than by you.

Memory per card

80 GB (H100) → 141 GB (H200) → 192 GB (B200) of HBM3e: fewer shards, larger micro-batches, fewer gradient-checkpoint recomputations.

Interconnect

NVLink 900 GB/s (H100) → 1.8 TB/s (B200), with NVL72 racks forming one 72-GPU domain: the all-reduce and shard-gather traffic of FSDP stops being the bottleneck.

7B

comfortable on one 8×H100 node with plain FSDP

70B

feasible on that node with ZeRO-3 + checkpointing; routine on a B200 node

Even so: for a bespoke taxonomy on thousands of labels, the encoder or QLoRA got you there for a fraction of this budget. Full fine-tuning is for genuine domain shift.

Reporting for Replication

01 Model name and exact revision hash, never just "Llama 3"

02 Quantisation config for training and for serving: NF4, INT8 or BF16

03 Full prompt text, temperature and decoding settings

04 LoRA rank, alpha, target modules, seed and epochs

05 Evaluation on a time-split held-out set, against a cheap baseline

06 Released adapters and label set in the replication package

An adapter is a few hundred megabytes: small enough to publish alongside the paper.

The Running Example, End to End

The path we walked:the earnings-call tone panel, from first API call to a node, each step entered by a measured failure.

Step	The move	Cost, time	Verdict
0	Hand-label 800 passages; fix the bar: $F1 \geq 0.85$, time split	a week of reading	no step without it
1 • API	Pinned Luna, temp 0, JSON schema, batch endpoint over ~1B tokens	~\$110, overnight	coarse tone: stop. Our bar: escalate
2 • Open weights	Llama-8B, pinned revision; token logprobs give a continuous tone index	~\$60 rented GPU, a day	stop if logprobs were the need, not accuracy
3 • Encoder FT	Fine-tune a small encoder on the 800 labels; score the corpus locally	one 24 GB card, overnight; corpus in ~40 min	most measurement papers end here
4 • LoRA	Analysis grows: extract guidance changes as structured JSON; QLoRA the 8B on the same labels	same card, ~3 h	stop: behaviour learned
5 • Full FT	New question: simulate manager Q&A; continued pretraining on 3B tokens of calls	8×H100 node, FSDP, a weekend	earned by a measured gap only

Rule:the ladder is a procedure: each step is entered by a measured failure of the step below, never by ambition.

The Ladder, Annotated

Experiment	Usual stop	Why
Simulated agents, vignettes	The API	Small N, capability-bound; the prompt is the treatment
Measurement, small public corpus	The API	Frontier model near the ceiling; token cost trivial
Text-as-data regressions	Open weights	Needs logprobs, embeddings and fixed citable weights
Confidential or licensed text	Open weights	Data cannot leave; the model comes to the enclave
Labelled classification at corpus scale	The encoder	With a few thousand labels, the 0.15B encoder wins the cost-accuracy trade; reruns are free
Bespoke taxonomy, domain shift	Fine-tune	Prompting fails held-out tests and labels exist: encoder first, then QLoRA
Anything, at first	The API	Always pilot on the API before building anything

Final Remarks

- 1 Start at the API and stop at the first step that passes a held-out test.
- 2 Climb for structural reasons: data governance, internals, scale or replication.
- 3 When you train, LoRA on one card covers almost every applied case; performance is task-dependent, so test on your task.
- 4 Report everything a referee needs to reproduce your results in five years: model version, weights, prompts, code and precision.